$$Cout_{FA} = \beta \cdot ab + C_{in}(ab + \beta) \equiv \left(\beta \rightarrow (a \rightarrow \bar{b})\right) \rightarrow \overline{\left(C_{in} \rightarrow \overline{\left((a \rightarrow \bar{b}) \rightarrow \beta\right)}\right)} \quad (9)$$

$$\mathbf{PPU_3}: Sum_{FA} = ab \oplus cd \oplus C_{in} \equiv \left[\left(\left(\overline{(a \rightarrow \bar{b})} \rightarrow \overline{(c \rightarrow \bar{d})}\right) \rightarrow \overline{\left((a \rightarrow \bar{b}) \rightarrow (c \rightarrow \bar{d})\right)}\right) \rightarrow C_{in}\right]$$
$$\rightarrow \overline{\left[\left(\left(\overline{(a \rightarrow \bar{b})} \rightarrow \overline{(c \rightarrow \bar{d})}\right) \rightarrow \overline{\left((a \rightarrow \bar{b}) \rightarrow (c \rightarrow \bar{d})\right)}\right) \rightarrow \overline{C_{in}}\right]} \quad (10)$$

$$Cout_{FA} = cd \cdot ab + C_{in}(ab + cd) \equiv \left(\overline{(c \rightarrow \bar{d})} \rightarrow (a \rightarrow \bar{b})\right) \rightarrow \overline{\left(C_{in} \rightarrow \overline{\left((a \rightarrow \bar{b}) \rightarrow \overline{(c \rightarrow \bar{d})}\right)}\right)} \quad (11)$$

The implementation algorithm of *PPU₂* and *PPU₃* are tabulated in Tables 10 and 11, respectively.

Table 10: The implementation algorithm of the proposed *PPU₂*.

| Step | Operation | Equivalent Logic |
|---|---|---|
| **1** | $S_1 = 0$ | $FALSE(S_1)$ |
| **2** | $S_2 = 0$ | $FALSE(S_2)$ |
| **3** | $S_3 = 0$ | $FALSE(S_3)$ |
| **4** | $(b \rightarrow 0) \equiv (b \rightarrow S_1) = S_1{}'$ | $S_1{}' = \bar{b}$ |
| **5** | $(a \rightarrow S_1{}') = S_1{}''$ | $S_1{}'' = \bar{a} + \bar{b}$ |
| **6** | $(S_1{}'' \rightarrow 0) \equiv (S_1{}'' \rightarrow S_3) = S_3{}'$ | $S_3{}' = ab = \alpha$ |
| **7** | $(\beta \rightarrow 0) \equiv (\beta \rightarrow S_2) = S_2{}'$ | $S_2{}' = \bar{\beta}$ |
| **8** | $(S_1{}'' \rightarrow \beta) = \beta'$ | $\beta' = ab + \beta$ |
| **9** | $(S_3{}' \rightarrow S_2{}') = S_2{}''$ | $S_2{}'' = (\bar{a} + \bar{b}) + \bar{\beta}$ |
| **10** | $S_3 = 0$ | $FALSE(S_3)$ |
| **11** | $(\beta' \rightarrow 0) \equiv (\beta' \rightarrow S_3) = S_3{}'$ | $S_3{}' = (\bar{a} + \bar{b}) \cdot \bar{\beta}$ |
| **12** | $(S_2{}'' \rightarrow S_3{}') = S_3{}''$ | $S_3{}'' = ab \cdot \beta + (\bar{a} + \bar{b}) \cdot \bar{\beta} = \overline{\alpha \oplus \beta}$ |
| **13** | $S_1 = 0$ | $FALSE(S_1)$ |
| **14** | $(C_{in} \rightarrow 0) \equiv (C_{in} \rightarrow S_1) = S_1{}'$ | $S_1{}' = \overline{C_{in}}$ |
| **15** | $(S_2{}'' \rightarrow C_{in}) = C_{in}{}'$ | $C_{in}{}' = ab \cdot \beta + C_{in}$ |
| **16** | $(S_3{}'' \rightarrow S_1{}') = S_1{}''$ | $S_1{}'' = (\alpha \oplus \beta) + \overline{C_{in}}$ |
| **17** | $S_3 = 0$ | $FALSE(S_3)$ |
| **18** | $(S_1{}'' \rightarrow 0) \equiv (S_1{}'' \rightarrow S_3) = S_3{}'$ | $S_3{}' = \overline{\alpha \oplus \beta} \cdot C_{in}$ |
| **19** | $S_1 = 0$ | $FALSE(S_1)$ |
| **20** | $(C_{in}{}' \rightarrow 0) \equiv (C_{in}{}' \rightarrow S_1) = S_1{}'$ | $S_1{}' = \left((\bar{a} + \bar{b}) + \bar{\beta}\right) \cdot \overline{C_{in}}$ |
| **21** | $(\beta' \rightarrow S_1{}') = S_1{}''$ | $S_1{}'' = (\bar{a} + \bar{b}) \cdot \bar{\beta} + \left((\bar{a} + \bar{b}) + \bar{\beta}\right) \cdot \overline{C_{in}}$ |
| **22** | $(\beta' \rightarrow C_{in}{}') = C_{in}{}''$ | $C_{in}{}'' = (\bar{a} + \bar{b}) \cdot \bar{\beta} + (ab \cdot \beta + C_{in})$ |
| **23** | $(C_{in}{}'' \rightarrow S_3{}') = S_3{}''$ | $S_3{}'' = \left[(\alpha + \beta) \cdot \left((\bar{\alpha} + \bar{\beta}) \cdot \overline{C_{in}}\right)\right] + (\overline{\alpha \oplus \beta} \cdot C_{in})$ $= (\alpha \oplus \beta \cdot \overline{C_{in}}) + (\overline{\alpha \oplus \beta} \cdot C_{in}) = \alpha \oplus \beta \oplus C_{in} = \boldsymbol{Sum}$ |
| **24** | $S_2 = 0$ | $FALSE(S_2)$ |
| **25** | $(S_1{}'' \rightarrow S_2) = S_2{}'$ | $S_2{}' = (\alpha + \beta) \cdot (\alpha \cdot \beta + C_{in}) = \alpha \cdot \beta + C_{in}(\alpha + \beta) = \boldsymbol{C_{out}}$ |

Table 11: The implementation algorithm of the proposed $PPU_3$.

| Step | Operation | Equivalent Logic |
|---|---|---|
| **1** | $S_1 = 0$ | $FALSE(S_1)$ |
| **2** | $S_2 = 0$ | $FALSE(S_2)$ |
| **3** | $(b \to 0) \equiv (b \to S_1) = S_1{}'$ | $S_1{}' = \bar{b}$ |
| **4** | $(a \to S_1{}') = S_1{}''$ | $S_1{}'' = \bar{a} + \bar{b}$ |
| **5** | $(d \to 0) \equiv (d \to S_2) = S_2{}'$ | $S_2{}' = \bar{d}$ |
| **6** | $(c \to S_2{}') = S_2{}''$ | $S_2{}'' = \bar{c} + \bar{d}$ |
| **7** | $S_3 = 0$ | $FALSE(S_3)$ |
| **8** | $S_4 = 0$ | $FALSE(S_4)$ |
| **9** | $(S_1{}'' \to 0) \equiv (S_1{}'' \to S_3) = S_3{}'$ | $S_3{}' = ab = \alpha$ |
| **10** | $(S_2{}'' \to 0) \equiv (S_2{}'' \to S_4) = S_4{}'$ | $S_4{}' = cd = \beta$ |
| **11** | $(S_1{}'' \to S_4{}') = S_4{}''$ | $S_4{}'' = ab + cd$ |
| **12** | $(S_3{}' \to S_2{}'') = S_2{}'''$ | $S_2{}''' = (\bar{a} + \bar{b}) + (\bar{c} + \bar{d})$ |
| **13** | $S_3 = 0$ | $FALSE(S_3)$ |
| **14** | $(S_4{}'' \to 0) \equiv (S_4{}'' \to S_3) = S_3{}'$ | $S_3{}' = (\bar{a} + \bar{b}) \cdot (\bar{c} + \bar{d})$ |
| **15** | $(S_2{}''' \to S_3{}') = S_3{}''$ | $S_3{}'' = ab \cdot cd + (\bar{a} + \bar{b}) \cdot (\bar{c} + \bar{d}) = \overline{\alpha \oplus \beta}$ |
| **16** | $S_1 = 0$ | $FALSE(S_1)$ |
| **17** | $(C_{in} \to 0) \equiv (C_{in} \to S_1) = S_1{}'$ | $S_1{}' = \overline{C_{in}}$ |
| **18** | $(S_2{}''' \to C_{in}) = C_{in}{}'$ | $C_{in}{}' = ab \cdot cd + C_{in}$ |
| **19** | $(S_3{}'' \to S_1{}') = S_1{}''$ | $S_1{}'' = (\alpha \oplus \beta) + \overline{C_{in}}$ |
| **20** | $S_3 = 0$ | $FALSE(S_3)$ |
| **21** | $(S_1{}'' \to 0) \equiv (S_1{}'' \to S_3) = S_3{}'$ | $S_3{}' = \overline{\alpha \oplus \beta} \cdot C_{in}$ |
| **22** | $S_2 = 0$ | $FALSE(S_2)$ |
| **23** | $(C_{in}{}' \to 0) \equiv (C_{in}{}' \to S_2) = S_2{}'$ | $S_2{}' = \left((\bar{a} + \bar{b}) + (\bar{c} + \bar{d})\right) \cdot \overline{C_{in}}$ |
| **24** | $(S_4{}'' \to S_2{}') = S_2{}''$ | $S_2{}'' = (\bar{a} + \bar{b}) \cdot (\bar{c} + \bar{d}) + \left((\bar{a} + \bar{b}) + (\bar{c} + \bar{d})\right) \cdot \overline{C_{in}}$ |
| **25** | $(S_4{}'' \to C_{in}{}') = C_{in}{}''$ | $C_{in}{}'' = (\bar{a} + \bar{b}) \cdot (\bar{c} + \bar{d}) + (ab \cdot cd + C_{in})$ |
| **26** | $(C_{in}{}'' \to S_3{}') = S_3{}''$ | $S_3{}'' = \left[(\alpha + \beta) \cdot \left((\bar{\alpha} + \bar{\beta}) \cdot \overline{C_{in}}\right)\right] + (\overline{\alpha \oplus \beta} \cdot C_{in})$ $= (\alpha \oplus \beta \cdot \overline{C_{in}}) + (\overline{\alpha \oplus \beta} \cdot C_{in}) = \alpha \oplus \beta \oplus C_{in} = \boldsymbol{Sum}$ |
| **27** | $S_4 = 0$ | $FALSE(S_4)$ |
| **28** | $(S_2{}'' \to S_4) = S_4{}'$ | $S_4{}' = (\alpha + \beta) \cdot (\alpha \cdot \beta + C_{in}) = \alpha \cdot \beta + C_{in}(\alpha + \beta) = \boldsymbol{C_{out}}$ |

The computational steps of the unsigned array multiplier based on the proposed PPUs' implementation algorithms are reduced compared to the classic structure. The total number of computational steps of the proposed n-bit unsigned CSA-based array multiplier is $25n^2 - 32n + 2$ which is obtained from (12). $CS_{UAmultiplier}$ refers to the number of computational steps of the unsigned array multiplier in (12). Moreover, *5n – 4* memristors (*2n* input and *3n – 4* work memristors) are needed to implement the proposed unsigned array multiplier.

$$CS_{UAmultiplier} = \#PPU_1 \times CS_{PPU1} + \#PPU_2 \times CS_{PPU2} + \#PPU_3 \times CS_{PPU3} + \#HA \times CS_{HA} + \#FA \times CS_{FA} + \#AND \times CS_{AND} \quad (12)$$

### 3.2 Proposed signed array multiplier

CSA-based array multiplier can also be applied to calculate the product of signed numbers. In an n-bit signed CSA-based array multiplier, PPUs are placed in columns *1* to *2n + 1* (in *n – 1* diagonal lines) and *n* rows. Figure 3 depicts the structure of a 4-bit signed array multiplier based on CSA. The main difference between signed and unsigned CSA-based array multipliers lies in their types of PPUs. As explained earlier, the unsigned CSA-based array multiplier consists of three types of PPUs, but the signed array multiplier comprises eight different PPU structures which are indicated with different colors in Figure 3.

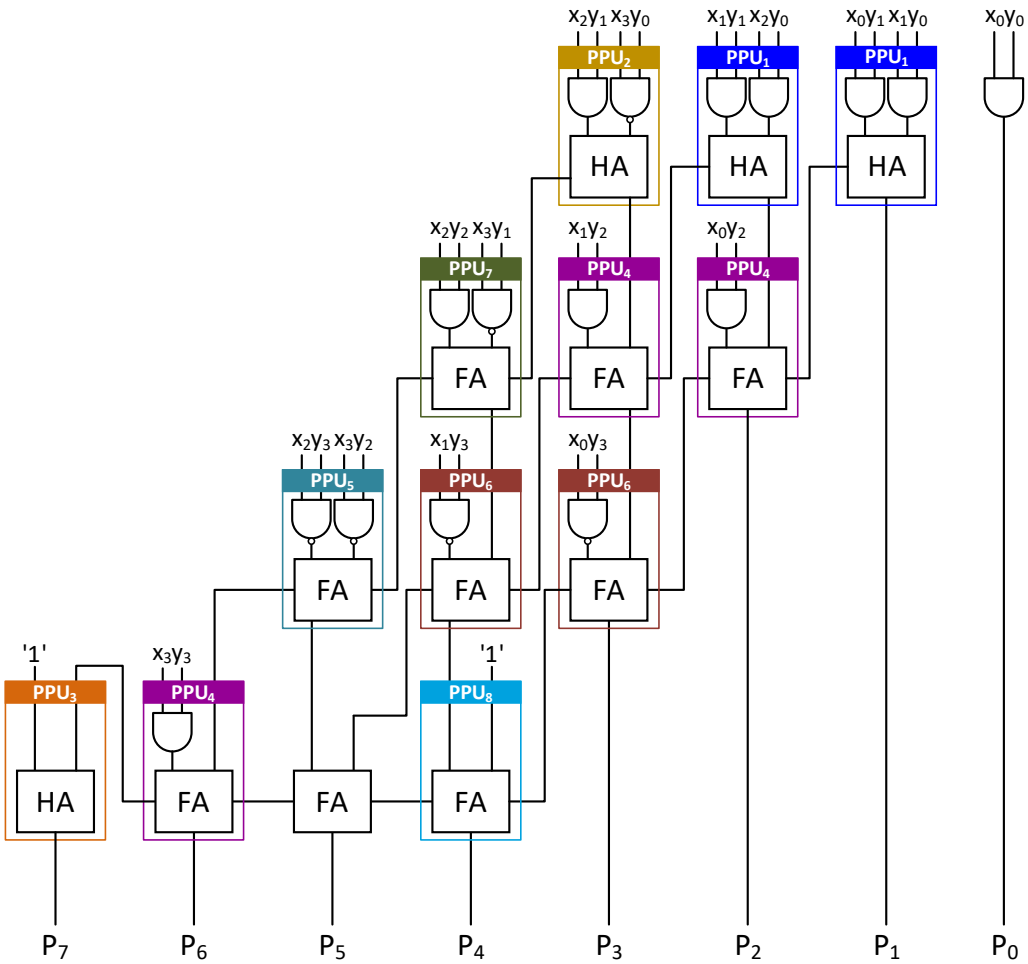


Figure 3: 4-bit signed CSA-based array multiplier.

In the signed CSA-based array multiplier, AND and NAND gates are applied to generate partial products (PPG stage). Half adder and full adder cells are also used to reduce the multiplication tree (PPR stage) and final addition (RCA stage). Eight types of PPUs form the signed CSA-based array multiplier containing different combinations of these logic gates and arithmetic cells. $PPU_1$ of the signed CSA-based array multiplier has the same structure as $PPU_1$ of the unsigned multiplier in which two inputs of the half adder cell are the outputs of two AND gates. In the structure of $PPU_2$, one of the inputs of the half adder cell is connected to the output of a NAND gate and the other to the output of an AND gate. There is only one $PPU_3$ cell in the structure of the signed array multiplier, which is a half adder located in the $2n^{th}$ column. One of the inputs of this PPU is a constant bit '1', and its other input is the $C_{out}$ of the full adder cell located in the $n^{th}$ row and $(2n - 1)^{th}$ column. In the $PPU_4$–$PPU_8$ structure, the full adder cell is applied to reduce the partial products and compute the final results in the RCA stage. In these five PPUs, one of the inputs of the full adder cell, located in the $i^{th}$ row and $j^{th}$ column, is the $C_{out}$ of the PPU located in the $(i - 1)^{th}$ row and $(j - 1)^{th}$ column. In $PPU_4$ and $PPU_6$, one of the inputs of the full adder cell (located in the $i^{th}$ row) is the *Sum* output of the half adder or